



سیستم‌های چندرسانه‌ای پیشرفته

گزارش تکلیف سوم

پوریا صامتی

۴۰۴۰۹۷۱۴

پائیز ۱۴۰۴

فهرست مطالب

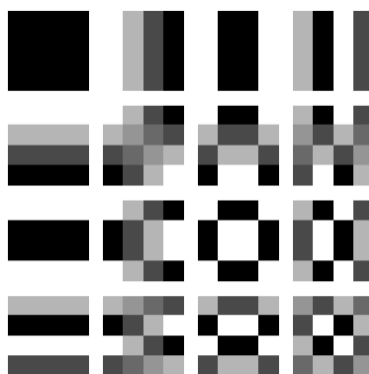
۳	پاسخ سوال اول
۵	پاسخ سوال دوم
۷	پاسخ سوال سوم
۷	رویکرد اول
۹	رویکرد دوم
۱۱	رویکرد سوم

پاسخ سوال اول

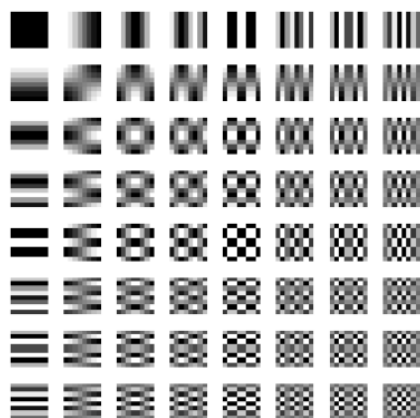
در این سوال قصد داریم تا ضرایب پایه تبدیل DCT دو بعدی را تولید کنیم. این تابع پایه‌های تبدیل را برای بلوک‌هایی با اندازه $N \times N$ تولید می‌کند؛ به‌طوری‌که برای هر جفت فرکانسی (u, v) یک تصویر پایه دوبعدی ساخته می‌شود. تابع با استفاده از فرمول استاندارد DCT، مقدار هر پیکسل (x, y) از تصویر پایه را به‌صورت حاصل‌ضرب دو تابع کسینوسی در جهت‌های افقی و عمودی محاسبه می‌کند و ضرایب نرمال‌سازی $\alpha(u)$ و $\alpha(v)$ را به‌کار می‌گیرد تا پایه‌ها متعامد و نرمال شوند. مؤلفه‌های با فرکانس پایین (مانند $u = 0$ یا $v = 0$) تغییرات آرام و یکنواخت را نمایش می‌دهند، در حالی که مؤلفه‌های با فرکانس بالاتر الگوهای سریع‌تر و جزئیات تصویر را مدل می‌کنند.

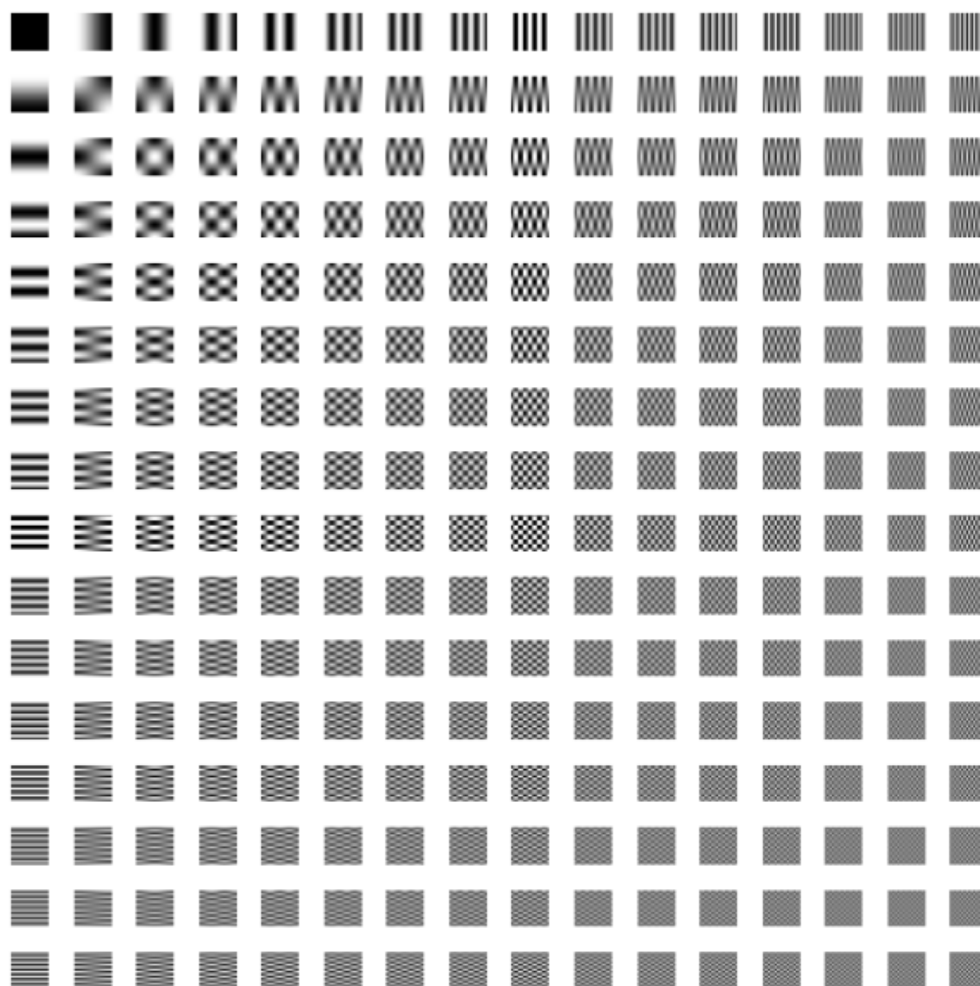
طبق خواسته سوال، این تصاویر پایه برای بلاک‌هایی به ابعاد زیر تولید شده‌اند:

بلاک 4x4:



بلاک 8x8:





همچنین توجه شود که برای محاسبه این ضرایب، از تابع زیر بهره گرفته شده است:

```
def generate_dct_2d_basis(N):
    basis_set = np.zeros((N, N, N, N), dtype=np.float64)

    for u in range(N):
        au = np.sqrt(1.0 / N) if u == 0 else np.sqrt(2.0 / N)

        for v in range(N):
            av = np.sqrt(1.0 / N) if v == 0 else np.sqrt(2.0 / N)

            for x in range(N):
                cos_x = np.cos((np.pi * (2*x + 1) * u) / (2*N))

                for y in range(N):
                    basis_set[u, v, x, y] = (au * av * cos_x * np.cos((np.pi * (2*y + 1) * v) / (2*N)))

    return basis_set
```

پاسخ سوال دوم

در این سؤال هدف آن است که با استفاده از تبدیل DCT و معکوس آن، فرآیند فشرده‌سازی تصویر پیاده‌سازی شود. در ابتدا قصد داشتیم مشابه سؤال اول، ضرایب DCT را مستقیماً با استفاده از محاسبه‌ی پایه‌ها به‌دست آوریم؛ اما از آنجا که این تبدیل باید به‌صورت یکپارچه روی کل تصویر اعمال می‌شد، پیچیدگی محاسباتی و به‌ویژه مصرف حافظه‌ی این روش بسیار بالا بود و با خطا مواجه می‌شدیم. به همین دلیل، برای محاسبه‌ی DCT و Inverse DCT از تجزیه‌ی ماتریسی استفاده کردیم و پیاده‌سازی تبدیل را بر اساس این رویکرد انجام دادیم که علاوه بر کاهش قابل‌توجه مصرف حافظه، از نظر محاسباتی نیز بهینه‌تر و مناسب‌تر برای پردازش تصاویر با ابعاد بزرگ است.:

```
def dct(image: np.ndarray, N: int):
    u = np.arange(N).reshape(-1,1)
    x = np.arange(N).reshape(1,-1)
    alpha = lambda k: np.sqrt(1/N) if k==0 else np.sqrt(2/N)

    C = np.zeros((N,N))
    for i in range(N):
        C[i,:] = alpha(i) * np.cos(np.pi*(2*x+1)*i / (2*N))

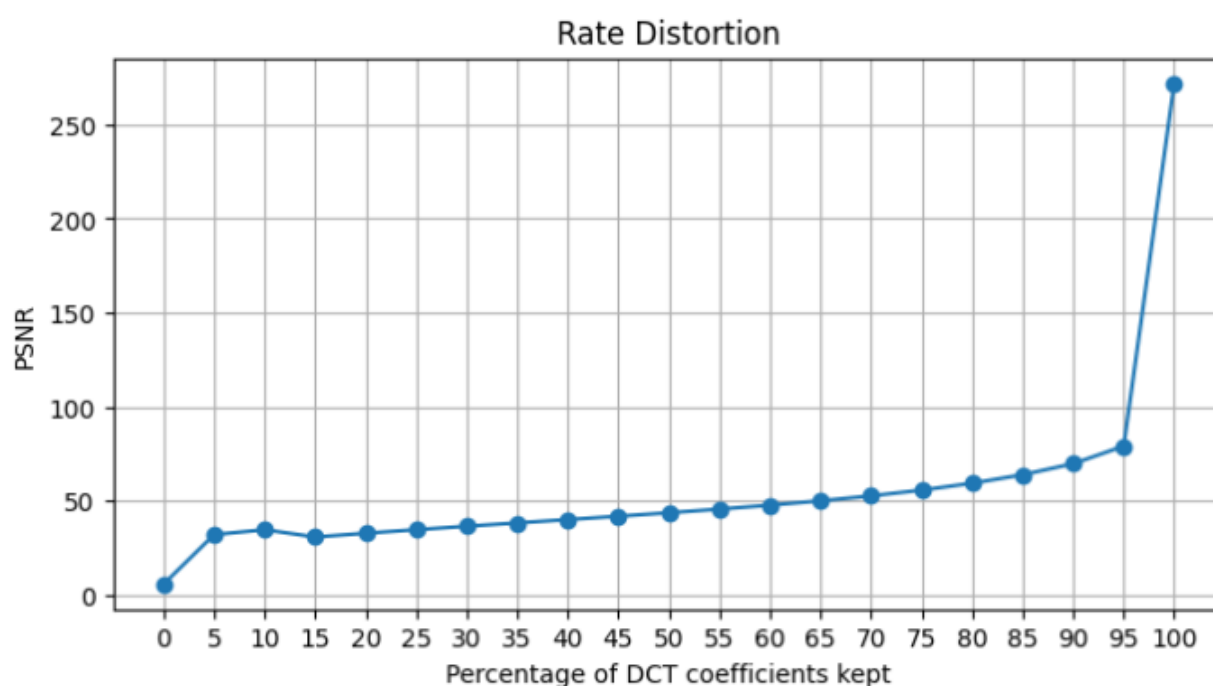
    dct_full = C @ image @ C.T
    return dct_full

def idct(dct_coef: np.ndarray, N: int):
    u = np.arange(N).reshape(-1,1)
    x = np.arange(N).reshape(1,-1)
    alpha = lambda k: np.sqrt(1/N) if k==0 else np.sqrt(2/N)

    C = np.zeros((N,N))
    for i in range(N):
        C[i,:] = alpha(i) * np.cos(np.pi*(2*x+1)*i / (2*N))

    image = C.T @ dct_coef @ C
    return image
```

در مرحله‌ی بعد، مطابق صورت سؤال، در هر تکرار تنها K درصد از ضرایب که دارای قدرمطلق بیشینه هستند، نگه داشته می‌شود و تصویر بازسازی‌شده صرفاً با استفاده از همین ضرایب منتخب تولید می‌گردد، در حالی که سایر ضرایب حذف می‌شوند. بدیهی است با کاهش درصد ضرایب نگه‌داشته‌شده، اطلاعات کمتری از تصویر اولیه در فرآیند بازسازی مورد استفاده قرار می‌گیرد و در نتیجه انتظار می‌رود کیفیت تصویر بازسازی‌شده کاهش یابد. این افت کیفیت به‌صورت کاهش مقدار PSNR نسبت به تصویر مرجع قابل مشاهده است. برای تحلیل کمی این رفتار، نمودار Rate-Distortion رسم شده است که رابطه‌ی بین نرخ (درصد ضرایب باقی‌مانده) و اعوجاج (PSNR) را نشان می‌دهد. نتیجه‌ی حاصل از این تحلیل در نمودار زیر نمایش داده شده است.



همچنین مطابق با خواسته سوال، تصاویر بازسازی شده باید ذخیره شوند. این تصاویر را در پوشه Barbara Reconstructed ذخیره شده اند.

پاسخ سوال سوم

در این سوال قصد داریم تا فشرده‌سازی را با استفاده از تبدیل فضای رنگی بررسی کنیم. قصد داریم تا از تبدیل RGB به YCbCr در حالت 4:2:0 استفاده کنیم. هدف نهایی آن است که روشی تهیه کنیم که PSNR تصویر بازسازی شده نسبت به تصویر اولیه دارای خطای کمتری باشد. در ادامه از ۳ رویکرد متفاوت بهره برده‌ایم.

رویکرد اول

در این رویکرد، به‌صورت ساده ابتدا در بخش Encoder تبدیل رنگ RGB به YCbCr با قالب 4:2:0 اعمال می‌شود. در این تبدیل کانال‌های Cb و Cr به‌صورت یکنواخت در هر دو جهت افقی و عمودی **Downsample** می‌شوند. در نتیجه، به ازای هر بلوک 2×2 از مؤلفه‌ی Y، تنها یک نمونه از Cb و یک نمونه از Cr ذخیره می‌گردد که دقیقاً متناظر با قالب 4:2:0 است. در بخش Decoder فرآیند معکوس انجام می‌شود؛ بدین معنا که ابتدا کانال‌های زیرنمونه‌برداری شده‌ی Cb و Cr به اندازه‌ی کانال Y باید **Upsample** شوند. در این پیاده‌سازی، این مرحله به‌صورت ساده با تکرار نمونه‌های مجاور انجام شده است تا ابعاد هر سه کانال یکسان گردد. پس از هم‌تراز شدن ابعاد، تبدیل معکوس YCbCr به RGB اعمال می‌شود و تصویر نهایی بازسازی می‌گردد.

```
def rgb2yCbCr_420(image: np.ndarray):
    H, W, _ = image.shape
    red, green, blue = image[:, :, 0], image[:, :, 1], image[:, :, 2]

    y = (0.299 * red) + (0.587 * green) + (0.114 * blue)
    Cb = (-0.168736 * red) + (-0.331264 * green) + (0.5 * blue) + 128
    Cr = (0.5 * red) + (-0.418688 * green) + (-0.081312 * blue) + 128

    Cb_420 = Cb[:, ::2, ::2]
    Cr_420 = Cr[:, ::2, ::2]

    return y, Cb_420, Cr_420

def yCbCr2rgb_420(YCbCr_image: np.ndarray):
    y, Cb_420, Cr_420 = YCbCr_image[0], YCbCr_image[1], YCbCr_image[2]
    H, W = y.shape

    Cb_up = np.repeat(np.repeat(Cb_420, 2, axis=0), 2, axis=1)
    Cr_up = np.repeat(np.repeat(Cr_420, 2, axis=0), 2, axis=1)

    Cb_up = Cb_up[:H, :W]
    Cr_up = Cr_up[:H, :W]

    red = y + (1.403 * (Cr_up - 128))
    green = y + (-0.344 * (Cb_up - 128)) + (-0.714 * (Cr_up - 128))
    blue = y + (1.773 * (Cb_up - 128))

    return np.stack([red, green, blue], axis=2)
```

نتایج اعمال این رویکرد به شرح زیر است:

÷	Image Name	÷	Red Channel	÷	Green Channel	÷	Blue Channel	÷	Average	÷
0	1.png		45.588614		58.662462		51.316886		51.855987	
1	2.png		43.725298		56.542978		47.409415		49.225897	
2	3.png		39.300166		51.334615		42.670117		44.434966	
3	4.png		34.305851		33.055501		33.884276		33.748543	
4	5.png		42.105021		55.490876		50.009712		49.201870	
5	6.png		44.504317		58.799619		40.835595		48.046510	
6	7.png		40.763066		52.829389		45.920946		46.504467	
7	8.png		40.853078		53.000295		45.870931		46.574768	
8	9.png		44.166106		51.601059		42.894389		46.220518	
9	Average		41.701280		52.368533		44.534696		46.201503	

در این رویکرد، Encoder دقیقاً مشابه با Encoder رویکرد اول می‌باشد. اما تفاوت در Decoder است. در رویکرد اول، ما در کانال Cr و Cb هر همسایه هر پیکسل را تکرار می‌کردیم تا فرآیند Upsampling انجام شود. این تغییر در تصویر، یک تغییر شدید می‌باشد. با توجه به همین امر، از یک فیلتر هموارسازی (Smoothing) استفاده کردیم. هدف ما این بود که شدت تغییرات را با یک فیلتر هموارسازی کاهش دهیم. به همین دلیل ابتدا Upsampling انجام دادیم. سپس با استفاده از سه فیلتر زیر هموارسازی را بررسی کردیم:

- فیلتر Average

- فیلتر Gaussian

- فیلتر Bilateral

در نتیجه پس از بررسی، به این نتیجه رسیدیم که استفاده از یک فیلتر هموارسازی گاوسی با کرنل به اندازه 3x3 باعث بهبود PSNR بین تمام تصاویر می‌شود. اعمال دو فیلتر دیگر نیز باعث بهبود نتایج شده بود اما تاثیر فیلتر گاوسی از بقیه بیشتر بود. به همین دلیل ساختار کلی Decoder در این رویکرد به شرح زیر است:

ابتدا دو کانال Cb و Cr باید Upsample شوند. سپس بر آنها باید فیلتر هموارسازی گاوسی با کرنل 3x3 اعمال کنیم.

```
def decoder_second(YCbCr_image: np.ndarray):
    y, Cb_420, Cr_420 = YCbCr_image[0], YCbCr_image[1], YCbCr_image[2]
    H, W = y.shape

    Cb_up = np.repeat(np.repeat(Cb_420, 2, axis=0), 2, axis=1)
    Cr_up = np.repeat(np.repeat(Cr_420, 2, axis=0), 2, axis=1)

    Cb_up = Cb_up[:H, :W]
    Cr_up = Cr_up[:H, :W]

    # Apply Gaussian smoothing filter using
    Cb_up = cv2.GaussianBlur(Cb_up, (3, 3), 0)
    Cr_up = cv2.GaussianBlur(Cr_up, (3, 3), 0)

    red = y + (1.403 * (Cr_up - 128))
    green = y + (-0.344 * (Cb_up - 128)) + (-0.714 * (Cr_up - 128))
    blue = y + (1.773 * (Cb_up - 128))

    return np.stack([red, green, blue], axis=2)
```

نتایج این روش، نسبت به رویکرد اول، پیشرفت قابل توجه را از خود نشان دادند. نتایج به شرح جدول زیر هستند:

÷	Image Name	÷	Red Channel	÷	Green Channel	÷	Blue Channel	÷	Average	÷
0	1.png		48.006413		60.610540		53.809773		54.142242	
1	2.png		45.601428		58.225423		49.237270		51.021374	
2	3.png		41.306310		53.382860		44.448479		46.379216	
3	4.png		36.001045		34.857536		35.457808		35.438796	
4	5.png		44.273369		57.501958		52.092069		51.289132	
5	6.png		46.942167		60.466525		43.937977		50.448890	
6	7.png		43.525813		55.433801		48.862350		49.273988	
7	8.png		43.548772		55.634869		48.702745		49.295462	
8	9.png		45.204929		52.777535		44.649211		47.543892	
9	Average		43.823361		54.321228		46.799742		48.314777	

رویکرد سوم

در این روش از پیشرفتی که در مرحله قبل انجام شد، بهره می‌بریم. همانطور که تغییرات شدید در حین بازسازی رخ میدهند و توانستیم اثر آنها را با یک فیلتر هموارسازی گاوسی کاهش دهیم، در این مرحله میخواهیم این تغییرات شدید را در حین انتقال فضا مدیریت کنیم (یعنی در Encoder تغییر ایجاد کنیم). در این رویکرد، Decoder همانند رویکرد دوم است. اما Encoder متفاوت است.

در این روش، ابتدا در بخش Encoder تصویر ورودی از فضای رنگی RGB به فضای YCbCr تبدیل می‌شود. سپس برای پیاده‌سازی دقیق قالب 4:2:0، کانال‌های رنگی Cb و Cr ابتدا به ابعاد زوج بریده می‌شوند و پس از آن با استفاده از میانگین‌گیری بلوکی 2×2 نمونه‌برداری می‌گردند؛ به این معنا که هر چهار پیکسل مجاور در این کانال‌ها با یک مقدار میانگین جایگزین می‌شوند که منجر به کاهش مؤثر وضوح مکانی رنگ بدون افت محسوس کیفیت ادراکی می‌شود. در Decoder، کانال‌های Cb و Cr نمونه‌برداری شده باید Upsample شوند و سپس برای کاهش آثار پله‌ای و ناپیوستگی‌های رنگی ناشی از بازسازی، یک فیلتر هموارساز گاوسی بر روی آن‌ها اعمال می‌گردد. در نهایت، با اعمال تبدیل معکوس YCbCr به RGB، تصویر بازسازی شده تولید می‌شود.

```
def encoder_third(image: np.ndarray):
    H, W, _ = image.shape
    red, green, blue = image[:, :, 0], image[:, :, 1], image[:, :, 2]

    y = (0.299 * red) + (0.587 * green) + (0.114 * blue)
    Cb = (-0.168736 * red) + (-0.331264 * green) + (0.5 * blue) + 128
    Cr = (0.5 * red) + (-0.418688 * green) + (-0.081312 * blue) + 128

    # Ensure even dimensions
    H2 = H - (H % 2)
    W2 = W - (W % 2)

    Cb = Cb[:H2, :W2]
    Cr = Cr[:H2, :W2]

    # 2x2 mean pooling (proper 4:2:0)
    Cb_420 = (Cb[0::2, 0::2] + Cb[0::2, 1::2] + Cb[1::2, 0::2] + Cb[1::2, 1::2]) / 4.0
    Cr_420 = (Cr[0::2, 0::2] + Cr[0::2, 1::2] + Cr[1::2, 0::2] + Cr[1::2, 1::2]) / 4.0

    return y, Cb_420, Cr_420
```

```
def decoder_third(YCbCr_image: np.ndarray):
    y, Cb_420, Cr_420 = YCbCr_image[0], YCbCr_image[1], YCbCr_image[2]
    H, W = y.shape

    Cb_up = np.repeat(np.repeat(Cb_420, 2, axis=0), 2, axis=1)
    Cr_up = np.repeat(np.repeat(Cr_420, 2, axis=0), 2, axis=1)

    Cb_up = Cb_up[:H, :W]
    Cr_up = Cr_up[:H, :W]

    # Apply Gaussian smoothing filter using
    Cb_up = cv2.GaussianBlur(Cb_up, (3, 3), 0)
    Cr_up = cv2.GaussianBlur(Cr_up, (3, 3), 0)

    red = y + (1.403 * (Cr_up - 128))
    green = y + (-0.344 * (Cb_up - 128)) + (-0.714 * (Cr_up - 128))
    blue = y + (1.773 * (Cb_up - 128))

    return np.stack([red, green, blue], axis=2)
```

نتایج این رویکرد بسیار بسیار پیشرفت خوبی را نسبت به دو رویکرد قبلی از خود نشان دادند. این نتایج به شرح زیر هستند:

÷	Image Name	÷	Red Channel	÷	Green Channel	÷	Blue Channel	÷	Average	÷
0	1.png		50.229907		62.755278		55.657460		56.214215	
1	2.png		47.524462		60.134482		51.541406		53.066783	
2	3.png		43.822160		55.898774		46.265788		48.662241	
3	4.png		31.884163		36.834960		37.686463		35.468529	
4	5.png		54.493737		61.217770		56.341304		57.350937	
5	6.png		49.155837		62.541526		46.197346		52.631570	
6	7.png		45.780332		57.694893		50.990617		51.488614	
7	8.png		46.937855		59.003269		52.044512		52.661879	
8	9.png		49.074750		56.498410		48.110333		51.227831	
9	Average		46.544800		56.953263		49.426137		50.974733	